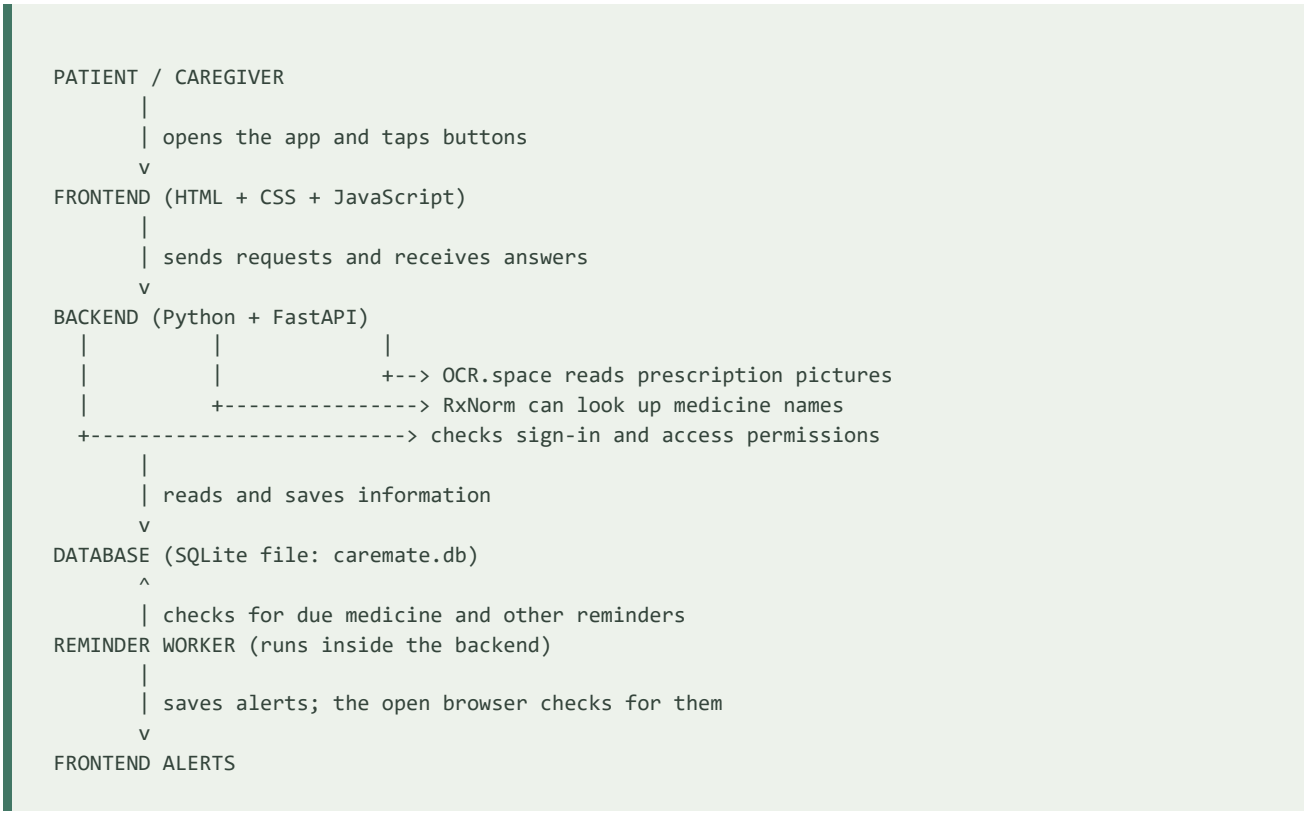
CareMate: How the App Fits Together

A simple guide to the parts of the app and how they work as one system

CareMate helps a patient keep track of medicines and lets an approved caregiver follow the care plan. The patient uses a **Caretaker** account in this app. A **Caregiver** account asks the patient for permission before it can see the patient's information.

This guide explains what each part does, where it lives, and how information moves between parts. It describes the current project files.

1. The app in one picture



The frontend is the part people see. The backend handles the rules and data. The database remembers information between visits. The reminder worker checks the database while the backend is running.

2. What the frontend does

The frontend is the set of files in `frontend/`. It runs in the user's web browser, and FastAPI serves it from the same address as the backend.

- **`frontend/index.html`** is the starting page. It loads the style and the JavaScript.
- **`frontend/app.css`** controls colors, spacing, buttons, cards, and how the page fits different screen sizes.
- **`frontend/app.js`** builds the screens, reads form entries, sends API requests, and updates the page with the answers.

For example, when someone marks a medicine as taken, the JavaScript sends the medicine and dose time to the backend. When the backend confirms the change, the page shows the new status.

The frontend does not directly open the database. It talks to the backend through web requests. This keeps the database work and access checks on the server.

3. What the backend does

The backend is written in Python and uses FastAPI. It receives requests from the browser, checks that they make sense, applies the app's rules, and returns a result.

- **`app.py` in the project folder** is a small start-up file. It points Uvicorn to the real app.
- **`backend/app.py`** contains the API routes. A route is an address for one job, such as signing in, listing medicines, or recording a dose. It also serves the frontend files.
- **`backend/security.py`** checks sign-in sessions and whether a user can access a patient's information.
- **`backend/database.py`** opens the SQLite database, creates missing tables, and updates older database layouts while keeping existing medicine and dose history.
- **`backend/schedule_generation.py`** suggests reminder times for some common frequency patterns. The user should review the suggested times.
- **`backend/medicine_extractor.py`** looks for possible medicine details in text read from a prescription.
- **`backend/ocr.py`** sends a prescription picture to OCR.space and receives the text it can read.
- **`backend/reminder_worker.py`** checks for due doses and other reminders and saves alerts for the patient and linked caregivers.
- **`backend/rxnorm.py`** supports medicine-name lookups. The API has an optional RxNorm lookup route.

The backend returns data to the browser, usually as JSON. JSON is a simple text format used to send information such as a medicine name, dose time, and status.

4. How the database remembers things

CareMate uses SQLite. Its database is the file **`caremate.db`** in the project folder. SQLite stores the app's information in tables, much like sheets in a workbook.

- **Users and sessions** store account details and sign-in sessions. Passwords are stored as protected hashes, not as readable passwords.
- **Patients** store patient profiles. A Caretaker account represents the patient and gets a profile when it is registered.
- **Caregiver requests and connections** store access requests and approved links between a patient and caregiver.
- **Medicines and medication logs** store medicine plans and whether a scheduled dose is pending, taken, missed, or skipped.
- **Prescriptions** store information about uploaded prescription images and text read from them.
- **Appointments, reminders, and notifications** store follow-ups and alerts for users to see.

When the backend starts, it checks that the needed tables are present. It can add missing parts to an older database. Keep `caremate.db`; deleting it would remove saved app information.

5. How the main parts work together

Sign-in and caregiver access

1. A person creates an account or signs in on the frontend.
2. The backend checks the account and returns a sign-in token.
3. The frontend sends that token with requests that need a signed-in user.
4. A Caregiver sends an access request to the patient's Caretaker account.
5. The patient approves or declines it. Only an approved caregiver is linked to the patient and allowed to view that patient's information.

Prescription to medicine plan

1. The user chooses a prescription picture in the frontend.
2. The backend sends the picture to OCR.space. This service tries to read the words in the image.
3. The medicine extractor looks for possible names and instructions in the returned text.
4. The frontend shows the possible details so the user can check them.
5. Only after the user reviews and saves the details does the backend create a medicine plan in SQLite.

OCR can read a prescription incorrectly. Check every detail against the original prescription. CareMate does not choose or change medicines for the user.

Schedule, dose, and report

1. The backend makes a schedule from saved medicine plans and their reminder times.
2. The frontend displays the scheduled doses.
3. When a user records a dose, the backend saves its status in the medication log.
4. The schedule and adherence report use those saved logs to show what happened.

6. How reminders reach the screen

The reminder worker starts with the backend. It checks saved medicine plans, dose logs, appointments, and custom reminders on a timer. When an alert is due, it saves a notification in SQLite for the patient and approved caregivers.

While the app is open, the browser checks the backend for new notifications and shows them in the app. The browser can also show a desktop alert if the user has allowed browser notifications.

The backend must be running for reminder checks to happen. The current app does not send SMS, email, or push alerts when it is closed. Browser alerts also require the app to be open and browser permission to be enabled.

7. How to start the connected app

From the `CareMate_ocr` folder, open PowerShell and run:

```
.\.venv\Scripts\Activate.ps1
uvicorn app:app --reload
```

Then open `http://127.0.0.1:8000/` in a browser. The same backend serves the frontend and its API. For interactive API documentation, open `http://127.0.0.1:8000/docs`.

Prescription reading needs an OCR.space API key in the `OCR_SPACE_API_KEY` environment variable. Other app features do not need that key.

8. Folder map and simple summary

```
CareMate_ocr/
|-- frontend/       Screens and browser behavior
|-- backend/        API, rules, security, database, and reminders
|-- scripts/legacy/ Older helper scripts kept from earlier versions
|-- caremate.db     Saved app information
|-- app.py          Starts the FastAPI app through Uvicorn
|-- requirements.txt Python packages needed by the app
|-- README.md       Setup and API notes
```

In short: **the browser shows the app, the backend does the work, the database remembers the information, and the reminder worker creates alerts.** They connect through web requests and run together when the FastAPI app is started.

CareMate is a tracking and reminder tool. It does not replace advice from a doctor or pharmacist.